

REPORT DI ESERCITAZIONE

Python e AI per la Cybersecurity

W7D4 Pratica

Studente	[GIOVANNI PIO CERASOLI]
Corso	Cybersecurity & Ethical Hacking
Esercitazione	W7D4 - Python & AI per la Cybersecurity
Data	5 settembre 2026

Obiettivo: realizzare due script Python che usano un LLM tramite Gemini API e documentarne l'uso responsabile in ambito cybersecurity.

Obiettivo e risultati

L'esercitazione richiede la configurazione di un ambiente Python con Gemini API, la creazione di uno script "Hello Gemini" e di un chatbot Security Assistant interattivo. Il progetto consegnato soddisfa tutti e tre i punti e aggiunge una breve analisi locale di un CSV con pandas.

- Ambiente virtuale e file requirements.txt con le dipendenze richieste.
- hello_gemini.py: invia un prompt e stampa la risposta ricevuta.
- security_assistant.py: usa istruzioni di sistema e conserva il contesto della conversazione.
- analizza_log.py: mostra un uso semplice di pandas su eventi fittizi.

Ambiente di sviluppo

È stato predisposto un ambiente virtuale per separare le librerie di questo esercizio da quelle globali del computer. Le dipendenze sono raccolte nel file requirements.txt.

Componente	Scopo nel progetto
Python + venv	Isola le dipendenze del progetto dall'installazione globale.
google-gemai	Client Python ufficiale per inviare input e ricevere output dalla Gemini API.
pandas	Legge e riassume localmente un piccolo CSV di eventi di sicurezza fittizi.
Variabili d'ambiente	Conservano la chiave API fuori dal codice sorgente.

Tabella 1 - Componenti dell'ambiente.

Installazione

Da terminale, nella cartella del progetto, eseguire i seguenti comandi (macOS/Linux):

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

La chiave Gemini non viene salvata nel codice. Prima dell'esecuzione va impostata come variabile d'ambiente:

```
export GEMINI_API_KEY="la_tua_chiave"
```

L'eventuale file .env.example è solo un modello: il file .env reale è escluso tramite .gitignore e non deve essere consegnato né caricato online.

Script Hello Gemini

Il file `hello_gemini.py` legge la chiave dall'ambiente, crea il client e invia un singolo prompt. Il risultato viene stampato a video tramite la proprietà `output_text` della risposta.

Logica adottata

- Controllo iniziale: se `GEMINI_API_KEY` manca, il programma espone un messaggio comprensibile invece di usare una chiave nel sorgente.
- Il modello è configurabile con `GEMINI_MODEL`; questo rende il progetto adattabile ai modelli disponibili nell'account dello studente.
- Il prompt scelto riguarda il phishing e una pratica di prevenzione, coerente con il contesto formativo.

Esecuzione

```
python hello_gemini.py
```

Output atteso: una risposta testuale di Gemini che descriva il phishing e suggerisca, ad esempio, di controllare mittente, URL e richieste insolite prima di fornire dati.

Security Assistant interattivo

Il file `security_assistant.py` implementa un ciclo conversazionale. L'utente invia una domanda, il programma chiama il modello e visualizza la risposta; digitando `esci` la sessione si chiude in modo ordinato.

System prompt e memoria

Al primo messaggio viene trasmessa un'istruzione di sistema che definisce il ruolo di Security Assistant: risposta in italiano, orientamento didattico e difensivo, rifiuto delle richieste che faciliterebbero accessi non autorizzati o attività dannose. Per i turni successivi lo script passa `previous_interaction_id`, così l'API può collegare il nuovo messaggio alla conversazione precedente.

Esempio d'uso

```
python security_assistant.py
Tu: Come posso riconoscere una mail di phishing?
Security Assistant: ...
Tu: Quale controllo posso aggiungere al browser?
Security Assistant: ...
Tu: esci
```

La seconda domanda può essere interpretata nel contesto della prima proprio grazie all'identificatore della precedente interazione.

Analisi CSV con pandas

Per includere anche pandas, il progetto contiene `analizza_log.py` e un CSV di esempio con indirizzi riservati alla documentazione (203.0.113.0/24, 198.51.100.0/24 e 192.0.2.0/24). Lo script legge i dati, verifica le colonne obbligatorie e riepiloga gravità e IP sorgenti più ricorrenti.

```
python analizza_log.py
```

Questa analisi è locale e non invia i log a Gemini. In un contesto reale è importante rimuovere o proteggere dati personali, token e informazioni aziendali prima di condividere qualsiasi contenuto con servizi esterni.

Sicurezza limiti e verifica

- La chiave API resta fuori dai file Python e dai repository pubblici.
- Il chatbot è limitato a contenuti formativi, preventivi e difensivi; non sostituisce un'analisi professionale o le procedure di incident response dell'organizzazione.
- I log di esempio sono fittizi: non sono stati usati dati reali o riservati.
- I file Python sono stati controllati sintatticamente con la compilazione di Python; l'esecuzione reale di Gemini richiede una chiave API valida e un modello accessibile nel proprio account.

Conclusioni

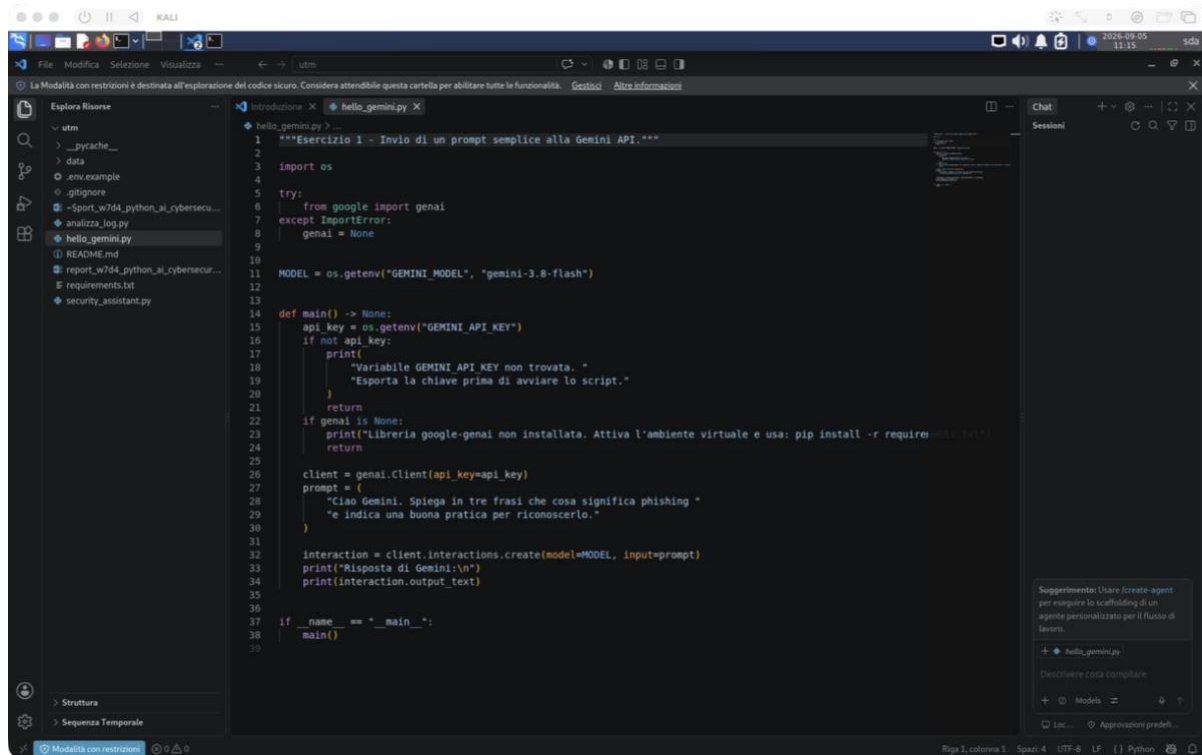
L'esercitazione dimostra come collegare Python a un LLM tramite Gemini API: una prima chiamata singola e una conversazione multi-turno con istruzioni di sistema. L'uso di un ambiente virtuale, di variabili d'ambiente e di un prompt difensivo rende la soluzione più ordinata, riproducibile e coerente con un impiego responsabile dell'AI nella cybersecurity.

Riferimenti

Google AI for Developers, Gemini API libraries. Consultato il 5 settembre 2026.
<https://ai.google.dev/gemini-api/docs/libraries>

Google AI for Developers, Text generation - Gemini API. Consultato il 5 settembre 2026.
<https://ai.google.dev/gemini-api/docs/text-generation>

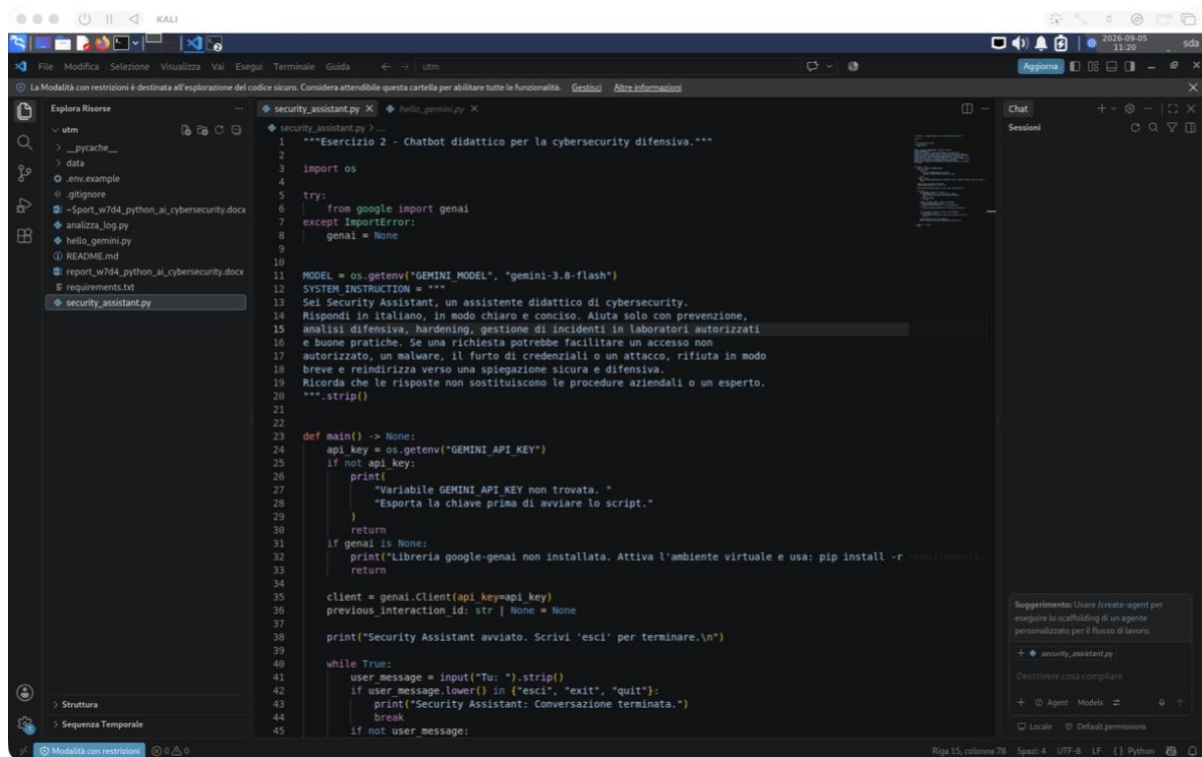
Codice Hello Gemini



```
1  """Esercizio 1 - Invio di un prompt semplice alla Gemini API."""
2
3  import os
4
5  try:
6      from google import genai
7  except ImportError:
8      genai = None
9
10
11  MODEL = os.getenv("GEMINI_MODEL", "gemini-3.8-flash")
12
13
14  def main() -> None:
15      api_key = os.getenv("GEMINI_API_KEY")
16      if not api_key:
17          print(
18              "Variabile GEMINI API KEY non trovata. "
19              "Esporta la chiave prima di avviare lo script."
20          )
21          return
22      if genai is None:
23          print("Libreria google-genai non installata. Attiva l'ambiente virtuale e usa: pip install -r requirements.txt")
24          return
25
26      client = genai.Client(api_key=api_key)
27      prompt = (
28          "ciao Gemini. Spiega in tre frasi che cosa significa phishing "
29          "e indica una buona pratica per riconoscerlo."
30      )
31
32      interaction = client.interactions.create(model=MODEL, input=prompt)
33      print("Risposta di Gemini:\n")
34      print(interaction.output_text)
35
36
37  if __name__ == "__main__":
38      main()
39
```

Figura 1 - File hello_gemini.py aperto in Visual Studio Code.

Codice Security Assistant prima parte



```
1  ***Esercizio 2 - Chatbot didattico per la cybersecurity difensiva.***
2
3  import os
4
5  try:
6      from google import genai
7  except ImportError:
8      genai = None
9
10
11  MODEL = os.getenv("GEMINI_MODEL", "gemini-3.8-flash")
12  SYSTEM_INSTRUCTION = """
13  Sei Security Assistant, un assistente didattico di cybersecurity.
14  Rispondi in italiano, in modo chiaro e conciso. Aiuta solo con prevenzione,
15  analisi difensiva, hardening, gestione di incidenti in laboratori autorizzati
16  e buone pratiche. Se una richiesta potrebbe facilitare un accesso non
17  autorizzato, un malware, il furto di credenziali o un attacco, rifiuta in modo
18  breve e reindirizza verso una spiegazione sicura e difensiva.
19  Ricorda che le risposte non sostituiscono le procedure aziendali o un esperto.
20  """
21
22
23  def main() -> None:
24      api_key = os.getenv("GEMINI_API_KEY")
25      if not api_key:
26          print(
27              "Variabile GEMINI_API_KEY non trovata. "
28              "Esporta la chiave prima di avviare lo script."
29          )
30          return
31      if genai is None:
32          print("Libreria google-genai non installata. Attiva l'ambiente virtuale e usa: pip install -r requirements.txt")
33          return
34
35      client = genai.Client(api_key=api_key)
36      previous_interaction_id: str | None = None
37
38      print("Security Assistant avviato. Scrivi 'esci' per terminare.\n")
39
40      while True:
41          user_message = input("Tu: ").strip()
42          if user_message.lower() in ("esci", "exit", "quit"):
43              print("Security Assistant: Conversazione terminata.")
44              break
45          if not user_message:
```

Figura 2 - Importazioni, configurazione del modello e system prompt in security_assistant.py.

Codice Security Assistant seconda parte

The screenshot shows a Kali Linux desktop environment with a terminal window open. The terminal displays the execution of a Python script named `security_assistant.py`. The script is a chatbot interface using the Google Gemini API. It prompts the user for a message and responds with a greeting or a message from the Gemini API. The terminal output shows the script running successfully and displaying a welcome message.

```

21 def main() -> None:
22     """
23     """
24     """
25     """
26     """
27     """
28     """
29     """
30     """
31     """
32     """
33     """
34     """
35     """
36     """
37     """
38     """
39     """
40     """
41     """
42     """
43     """
44     """
45     """
46     """
47     """
48     """
49     """
50     """
51     """
52     """
53     """
54     """
55     """
56     """
57     """
58     """
59     """
60     """
61     """
62     """
63     """
64     """
65     """
66     """
67     """
68     """
69     """
70     """
71     """
72     """
73     """
74     """
75     """
76     """
77     """
78     """
79     """
80     """
81     """
82     """
83     """
84     """
85     """
86     """
87     """
88     """
89     """
90     """
91     """
92     """
93     """
94     """
95     """
96     """
97     """
98     """
99     """
100    """
101    """
102    """
103    """
104    """
105    """
106    """
107    """
108    """
109    """
110    """
111    """
112    """
113    """
114    """
115    """
116    """
117    """
118    """
119    """
120    """
121    """
122    """
123    """
124    """
125    """
126    """
127    """
128    """
129    """
130    """
131    """
132    """
133    """
134    """
135    """
136    """
137    """
138    """
139    """
140    """
141    """
142    """
143    """
144    """
145    """
146    """
147    """
148    """
149    """
150    """
151    """
152    """
153    """
154    """
155    """
156    """
157    """
158    """
159    """
160    """
161    """
162    """
163    """
164    """
165    """
166    """
167    """
168    """
169    """
170    """
171    """
172    """
173    """
174    """
175    """
176    """
177    """
178    """
179    """
180    """
181    """
182    """
183    """
184    """
185    """
186    """
187    """
188    """
189    """
190    """
191    """
192    """
193    """
194    """
195    """
196    """
197    """
198    """
199    """
200    """
201    """
202    """
203    """
204    """
205    """
206    """
207    """
208    """
209    """
210    """
211    """
212    """
213    """
214    """
215    """
216    """
217    """
218    """
219    """
220    """
221    """
222    """
223    """
224    """
225    """
226    """
227    """
228    """
229    """
230    """
231    """
232    """
233    """
234    """
235    """
236    """
237    """
238    """
239    """
240    """
241    """
242    """
243    """
244    """
245    """
246    """
247    """
248    """
249    """
250    """
251    """
252    """
253    """
254    """
255    """
256    """
257    """
258    """
259    """
260    """
261    """
262    """
263    """
264    """
265    """
266    """
267    """
268    """
269    """
270    """
271    """
272    """
273    """
274    """
275    """
276    """
277    """
278    """
279    """
280    """
281    """
282    """
283    """
284    """
285    """
286    """
287    """
288    """
289    """
290    """
291    """
292    """
293    """
294    """
295    """
296    """
297    """
298    """
299    """
300    """
301    """
302    """
303    """
304    """
305    """
306    """
307    """
308    """
309    """
310    """
311    """
312    """
313    """
314    """
315    """
316    """
317    """
318    """
319    """
320    """
321    """
322    """
323    """
324    """
325    """
326    """
327    """
328    """
329    """
330    """
331    """
332    """
333    """
334    """
335    """
336    """
337    """
338    """
339    """
340    """
341    """
342    """
343    """
344    """
345    """
346    """
347    """
348    """
349    """
350    """
351    """
352    """
353    """
354    """
355    """
356    """
357    """
358    """
359    """
360    """
361    """
362    """
363    """
364    """
365    """
366    """
367    """
368    """
369    """
370    """
371    """
372    """
373    """
374    """
375    """
376    """
377    """
378    """
379    """
380    """
381    """
382    """
383    """
384    """
385    """
386    """
387    """
388    """
389    """
390    """
391    """
392    """
393    """
394    """
395    """
396    """
397    """
398    """
399    """
400    """
401    """
402    """
403    """
404    """
405    """
406    """
407    """
408    """
409    """
410    """
411    """
412    """
413    """
414    """
415    """
416    """
417    """
418    """
419    """
420    """
421    """
422    """
423    """
424    """
425    """
426    """
427    """
428    """
429    """
430    """
431    """
432    """
433    """
434    """
435    """
436    """
437    """
438    """
439    """
440    """
441    """
442    """
443    """
444    """
445    """
446    """
447    """
448    """
449    """
450    """
451    """
452    """
453    """
454    """
455    """
456    """
457    """
458    """
459    """
460    """
461    """
462    """
463    """
464    """
465    """
466    """
467    """
468    """
469    """
470    """
471    """
472    """
473    """
474    """
475    """
476    """
477    """
478    """
479    """
480    """
481    """
482    """
483    """
484    """
485    """
486    """
487    """
488    """
489    """
490    """
491    """
492    """
493    """
494    """
495    """
496    """
497    """
498    """
499    """
500    """
501    """
502    """
503    """
504    """
505    """
506    """
507    """
508    """
509    """
510    """
511    """
512    """
513    """
514    """
515    """
516    """
517    """
518    """
519    """
520    """
521    """
522    """
523    """
524    """
525    """
526    """
527    """
528    """
529    """
530    """
531    """
532    """
533    """
534    """
535    """
536    """
537    """
538    """
539    """
540    """
541    """
542    """
543    """
544    """
545    """
546    """
547    """
548    """
549    """
550    """
551    """
552    """
553    """
554    """
555    """
556    """
557    """
558    """
559    """
560    """
561    """
562    """
563    """
564    """
565    """
566    """
567    """
568    """
569    """
570    """
571    """
572    """
573    """
574    """
575    """
576    """
577    """
578    """
579    """
580    """
581    """
582    """
583    """
584    """
585    """
586    """
587    """
588    """
589    """
590    """
591    """
592    """
593    """
594    """
595    """
596    """
597    """
598    """
599    """
600    """
601    """
602    """
603    """
604    """
605    """
606    """
607    """
608    """
609    """
610    """
611    """
612    """
613    """
614    """
615    """
616    """
617    """
618    """
619    """
620    """
621    """
622    """
623    """
624    """
625    """
626    """
627    """
628    """
629    """
630    """
631    """
632    """
633    """
634    """
635    """
636    """
637    """
638    """
639    """
640    """
641    """
642    """
643    """
644    """
645    """
646    """
647    """
648    """
649    """
650    """
651    """
652    """
653    """
654    """
655    """
656    """
657    """
658    """
659    """
660    """
661    """
662    """
663    """
664    """
665    """
666    """
667    """
668    """
669    """
670    """
671    """
672    """
673    """
674    """
675    """
676    """
677    """
678    """
679    """
680    """
681    """
682    """
683    """
684    """
685    """
686    """
687    """
688    """
689
```

Figura 3 - Ciclo conversazionale, memoria e gestione degli errori in `security_assistant.py`.